Homework 2 — Will the projectile clear the wall?

Scientific Programming for Physicists | Week 2 | Main topic: conditionals (`if`, `elseif`, `else`)

Goal

In this exercise you will write a complete Julia program: it reads five numbers from the keyboard, checks that they satisfy the model's requirements, calculates the projectile's height at the wall, and prints the result. Complete the input commands, calculations, conditionals, and output commands yourself. Depending on the outcome, the program prints either one or three result lines. No function definitions or loops are needed.

Physical model

A projectile is launched from ground level with initial speed `speed_m_s` at an angle `angle_deg` above the horizontal. A wall is a horizontal distance `wall_x_m` away and has height `wall_height_m`. The magnitude of gravitational acceleration is `gravity_m_s2`. The positive horizontal direction points toward the wall; the positive vertical direction points upward. Assume level ground, constant gravity, and no air resistance.

In the following formulas, v_0 is the launch speed, θ is the launch angle, and g is the magnitude of gravitational acceleration. Calculate:

- Horizontal velocity: $v_x = v_0 \cos(\theta)$
- Time to reach the wall plane: $t = \text{wall_x_m} / v_x$
- Calculated height at the wall plane: $\text{height_at_wall} = v_0 \sin(\theta)\, t - g\, t^2 / 2$
- Vertical velocity at the wall plane: $v_y = v_0 \sin(\theta) - g\, t$

You may use `sind` and `cosd`, or convert the angle to radians with `deg2rad`.

The calculated `height_at_wall` follows the *mathematical* continuation of the trajectory, even if the projectile has already landed before the wall. A negative height means that it landed before the wall. In that case, print only the landing message; there is no need to calculate the vertical velocity at the wall plane. You do not need to simulate a collision or calculate the landing time separately.

Input

Read five numbers, one per line, in this order:

Order	Variable	Meaning	Units
1	`speed_m_s`	Launch speed	m/s
2	`angle_deg`	Launch angle	degrees
3	`wall_x_m`	Horizontal wall distance	m
4	`wall_height_m`	Wall height	m
5	`gravity_m_s2`	Gravity magnitude	m/s²

```
Launch speed (m/s):
Launch angle (deg):
Wall distance (m):
Wall height (m):
Gravity (m/s^2):
```

Reading each value combines two operations: `readline()` reads a line of text, and `parse(Float64, ...)` converts it to a `Float64` number. The first input is already provided in the template:

```
println("Launch speed (m/s):")
speed_m_s = parse(Float64, readline())
```

Complete the input commands for the other four values in the same way. For each value, first print the prompt from the table using `println`, then read the number. Each prompt should appear on its own line.

For this exercise, assume that each line contains a number in a format that `parse` can convert. You do not need to handle text such as `hello`, missing lines, or repeated input requests after an error.

Allowed input values

All five values must be finite. In addition:

- `speed_m_s > 0`
- `0 <= angle_deg < 90`
- `wall_x_m >= 0`
- `wall_height_m >= 0`
- `gravity_m_s2 > 0`

Before dividing, also check that the horizontal velocity is positive: `vx > 0`.

Because input is converted with `parse(Float64, ...)`, every successfully read value is a `Float64`. Use `isfinite` and check the ranges above. No additional type checks, such as `isa Real` or checking for `Bool`, are required.

If a value is invalid, stop the program with `throw(ArgumentError("..."))` **before calculating the arrival time, height, or vertical velocity, and before printing any result lines**. Input prompts may already have been printed. Choose your own error-message wording.

Result variables

For every valid input, calculate `height_at_wall`, the height at the wall plane in meters.

If `height_at_wall < 0`, print the landing message below. You do not need to define `vertical_state` or `clears` in this case.

Otherwise, when `height_at_wall >= 0`, also calculate the vertical velocity `vy` and store the following results:

- `vertical_state` — the string `"rising"` if `vy > 0`, otherwise `"falling"`. By the exercise's convention, assign `"falling"` when `vy == 0` as well.
- `clears` — a Boolean: `true` if `height_at_wall >= wall_height_m`, otherwise `false`. This variable indicates whether the calculated height reaches at least the wall's height. An exact hit on the top edge therefore gives `true`, although the output has a separate message for that case.

Output

The projectile lands before the wall

If `height_at_wall < 0`, print **only one result line**:

```
Result: The projectile lands before the wall.
```

Do not print the `Height at wall` or `Vertical motion` lines in this case.

The projectile reaches the wall position at a nonnegative height

If `height_at_wall >= 0`, print **exactly three result lines**:

```
Height at wall: <number> m
Vertical motion: <state>
Result: <result>
```

The first line shows your calculated height. Julia's normal numerical formatting is acceptable; the checked value may differ from the expected height by at most `0.0001` meters. You may round the displayed value to four decimal places, but use the original, unrounded height for all conditionals.

In the second line, `<state>` must be exactly `rising` or `falling`.

The third line must match exactly one of these three messages:

Condition	Third output line
`0 <= height_at_wall < wall_height_m`	`Result: The projectile hits the wall.`
`height_at_wall == wall_height_m`	`Result: The projectile hits the edge of the wall.`
`height_at_wall > wall_height_m`	`Result: The projectile clears the wall.`

Use an outer conditional to distinguish landing before the wall from reaching the wall position at a nonnegative height. In the second branch, use nested conditionals to determine the vertical state and classify the wall interaction. Keep the height and vertical-motion output inside that branch.

Print only the input prompts and the result lines required for the case. Do not print headings, extra messages, or intermediate values.

For an exact hit on the top edge, use == on the unrounded values. The equality case in the self-check table uses zero wall distance and zero wall height, giving exact equality. You do not need to add a tolerance to this equality test.

For `speed_m_s = 20.0, angle_deg = 45.0, wall_x_m = 10.0, wall_height_m = 7.0,` and `gravity_m_s2 = 9.81`, the three result lines are:

```
Height at wall: 7.5475 m
Vertical motion: rising
Result: The projectile clears the wall.
```

Self-checking

Run the following six cases. Use `gravity_m_s2 = 9.81` in every case. The height and vertical-state columns describe the expected output; heights are in meters. A dash means that the line is not printed. The abbreviated outcomes refer to the full messages in the Output section.

speed_m_s	angle_deg	wall_x_m	wall_height_m	Height (m)	State	Result
20	45	10	7	7.5475	rising	clears
20	45	10	8	7.5475	rising	hits wall
20	45	30	7	7.9275	falling	clears
20	45	30	8	7.9275	falling	hits wall
20	45	50	2	—	—	lands before wall
20	0	0	0	0.0	falling	hits edge

In the landing-before-the-wall case, the calculated height is approximately `-11.3125` meters. Use it to check the condition, but do not print it. The only result line is the landing message.

The last case checks exact equality in height. It is physically artificial but useful as a boundary check.

Also check invalid inputs: zero or negative speed, a negative angle or an angle of `90` degrees or more, negative wall distance, negative wall height, zero or negative gravity, and a non-finite value such as `Inf` or `NaN`. In each run, the program should stop with `ArgumentError` without printing any result lines.

Automatic checker

Keep `hw2_checker.jl` together with your completed file, in the same folder. Run:

```
julia hw2_checker.jl YOUR_ID.jl
```

The checker starts a fresh Julia process for each case and supplies five numbers through standard input. Your ordinary `readline()` calls receive those numbers, so your program needs no testing code, special command-line handling, or marker lines. The checker does not modify your file. It checks the six cases above and 18 invalid inputs, including `Inf` and `NaN`, and prints PASS/FAIL for each case. On failure it shows the input, the expected result, and your program's actual output. Passing all checks is strong evidence that the main branches work; it does not prove correctness for every possible input. The checker uses only Julia's standard library.

Run `julia YOUR_ID.jl` separately to try your own values.

Working and submitting

1. Put `HW2_student_template.jl` and `hw2_checker.jl` in the same folder, and save the template as `YOUR_ID.jl`.
2. Complete the input commands for the four missing values, then write the calculations, conditionals, and output commands in that file.
3. Run `julia hw2_checker.jl YOUR_ID.jl`. Correct every failing case, and use `julia YOUR_ID.jl` to try your own values.
4. Submit **only your completed file** on the course Moodle page.

The template and checker use no extra packages. File reading within your program, function definitions, and loops are not required.